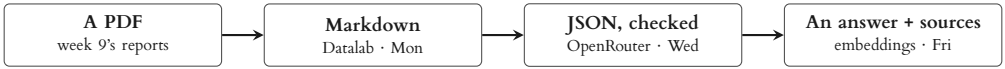
AI APIs as Data Tools

Week 13 · APIs module · Textbook Chapter 13

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

November 9, 11 & 13, 2026

This week at a glance



One pipeline on Boulder's revenue reports from week 9, every class live coding, from one lab notebook in three parts. No chatbots: one request in, one JSON answer out, and every answer checked.

Monday | Lab, part 1: document understanding

- A PDF, picture of a table included, to Markdown; then week 9's sums

Wednesday | Lab, part 2: a model as a function

- One page in, JSON in our shape out, checked against week 9

Friday | Lab, part 3: search by meaning; then your project

- Embeddings, an answer with sources or null, MCP and skills; 15 minutes on checkpoint 3

Two free keys, before Monday

`DATALAB_KEY` from datalab.to, and `OPENROUTER_KEY` from openrouter.ai. Set each as an environment variable, close the terminal, open a new one, and start Jupyter from there.

Due Sunday, Nov 15

Chapter 13's **Recommended Exercises**, from its companion notebook, and portfolio checkpoint 3, *Analyze your data*. Graded on good-faith attempt to complete rather than getting everything working.

1. Monday · Live-Coding Lab

Daily note questions

- **Question from the daily note?** — Answer, in one line.

This week, every class is live coding

Find it | `labs/` in the course repository

1. Open the **labs** folder of `cuinfoscience/INF04617-Fall2026` on GitHub
2. Open `week-13`, click `lab-13.ipynb`, then **Download raw file**
3. Open it in JupyterLab in the `webdata` environment

Use it | one part a day

- I write the code on screen, and you write it in your copy
- Put **your** email in the User-Agent in Step 0
- Today: Part 1, Steps 0–5, with both keys set
- Wednesday and Friday: download the new copy and start at that part's setup step

Solutions

`lab-13-key.ipynb`, beside it: every cell filled in and run. Fall behind? Copy the cell from the key and keep going.

Nothing to submit

The lab is practice. After each step, a **Discuss** box holds questions we'll talk over before the next.

Error flavors

A `429` from a free model means the shared pool is busy: wait, and send it again. A `KeyError` on `"choices"` means the answer was an error: print it and look. A request that fails for everyone is *broken*: tell me, or open an issue.

Models as functions, not chatbots

A chatbot answers a person. This week a model answers a program: one request in, one answer out, in a shape we chose, and checked before we use it.

Three kinds of AI API, one a day:

- **Document understanding**: a PDF in, Markdown out, pictures of tables included (Datalab)
- **Structured output**: a page in, JSON in our schema out (OpenRouter)
- **Embeddings**: text in, a list of numbers out, so we can search by meaning (OpenRouter)

Why check every answer? A model's numbers look right whether or not they are.

What you send, and to whom

Every page we send goes to a company: Datalab, or OpenRouter and whoever runs the model. Boulder's reports are public records that name no private individuals. Send nothing you couldn't publish.

Page 1 stays unprinted

Each report's cover gives a city staff member's email and phone. We never print page 1, as week 9 blacked it out.

Step 0: two keys

1. **Datalab**: sign up at datalab.to. Copy a key from API Keys. The free allowance needs no card.
2. **OpenRouter**: sign up at openrouter.ai. Create a key at Keys. Models whose ID ends in `:free` cost nothing.
3. Store both with the `webdata` environment, in a terminal (on Windows, the Anaconda Prompt):

```
conda env config vars set DATALAB_KEY="your-key" -n webdata
conda env config vars set OPENROUTER_KEY="your-key" -n webdata
```

Then close that terminal, open a new one, run `conda activate webdata`, and start Jupyter from it. Datalab's docs call its key `DATALAB_API_KEY`; we use the course's naming, as with `CENSUS_KEY`.

```
print("DATALAB_KEY:", "set"
      if os.environ.get("DATALAB_KEY")
      else "MISSING")
print("OPENROUTER_KEY:", "set"
      if os.environ.get("OPENROUTER_KEY")
      else "MISSING")
# DATALAB_KEY: set
# OPENROUTER_KEY: set
```

It's a password

Never paste a key into a notebook cell or a code file. `MISSING`? Quit Jupyter and start it from a new terminal; or Step 0's given cell asks with `getpass()`.

Step 1: submit a document, get a ticket

```
datalab_base = "https://www.datalab.to/api/v1"
book = ("https://raw.githubusercontent.com/cuinfoscience/"
        "Web-Data-Science-Book/main/data/ch-09/")
data = {"file_url": book + "revenue-report-2024-12.pdf",
        "output_format": "markdown",
        "mode": "balanced", "paginate": "true"}
job = requests.post(f"{datalab_base}/convert", data=data,
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(job)
```

The answer isn't the document. Converting takes seconds, so Datalab answers at once with a **ticket**: `request_check_url`, the address where the result will be.

Docs: Datalab's Convert API

The ticket's fields

Field	Holds
`success`	`True` if it was accepted
`error`	why not, if not
`request_id`	the job's ID
`request_check_url`	where to ask

From Datalab's API reference.

An address, not a file

We send `file_url`, the book's copy of week 9's PDF, so Datalab fetches it. A file only on your laptop goes as an upload instead.

Step 2: wait for the result

```
check = requests.get(job["request_check_url"],
    headers={"X-API-Key": datalab_key, **HEADERS}).json()
print(check["status"])

# Given: ask every two seconds until it's done
while check["status"] != "complete":
    time.sleep(2)
    check = requests.get(job["request_check_url"],
        headers={"X-API-Key": datalab_key,
                 **HEADERS}).json()
print(check["page_count"], "pages,", check["total_cost"],
      "cents,", round(check["runtime"], 1), "seconds")
# 15 pages, 4.5 cents, 10.2 seconds
```

`status` is `processing` until it's `complete`, as week 12's loop followed the `cursor` until there wasn't one.

Docs: the result check

What it cost

Measured on November 7:

- All seven year-end reports, `balanced`: 106 pages for 31.8 cents, 12–15 seconds each
- This report in `lite` mode: 0.56 cents, with rougher tables
- The free allowance: about 3,000 pages a month

Discuss

The loop has no limit. If Datalab never finished, what would your notebook do?

Step 3: read the Markdown, page by page

```
markdown = check["markdown"]
pages = re.split(r"\n*\{\d+\}-{10,}\n*",
                 markdown)[1:]
print(len(pages), "pages")
# 15 pages
print(pages[1][:1200])   # the summary
print(pages[14][:1500])  # Exhibit 3
```

With `paginate`, each page starts with a marker such as `{14}` and a line of dashes. Split on it, and drop the empty piece before the first.

Never `pages[0]`

The cover gives a staff member's email and phone. So does Exhibit 3's note: we cut it below.

Exhibit 3, a picture of a table, as Markdown

```
## Exhibit 3: Sales and Use Tax by Month

(the note: phone the city for "a screen
readable version"; its number cut here)

| REVENUE<br>CATEGORY      | YEAR | JAN
|--------------------------|------|----------
|                          |      |
| <b>Retail Sales Tax</b>  | 2019 | 8,590,452
|                          | 2020 | 7,761,028
|                          | 2021 | 8,059,343
...
```

Week 9's pdfplumber stopped here: a picture has no text layer. Datalab read all 30 rows of figures.

Chart 8, page 9: a pie chart, figures printed on its slices

Category	Amount
State Share-Back	$529,220
Medical Marijuana (3.86% Base)	$52,937
Recreational Marijuana (3.86% Base)	$1,262,688
Recreational Marijuana (3.5% Additional + 5.0% Excise)	$1,341,007

Under the table, Datalab's description:
"The largest slice is Recreational Marijuana (3.86% Base) at $1,262,688, followed by Recreational Marijuana (3.5% Additional + 5.0% Excise) at $1,341,007."

The table was **read**. The sentence was **written**, and its own figures contradict it.

Page 10: a bar chart with no numbers

The sugar-sweetened beverage chart prints bars and an axis, and no figures. Datalab wrote a table beside it anyway: 72 values, one per month for six years, each with a `~`.

December 2024: `~$285,000`. That number appears nowhere in the PDF.

Discuss

What else on these pages did a model write rather than read? How would you tell which is which?

Step 4: check what it read

```
checked = 0
for line in pages[14].splitlines():
    cells = [c.strip().replace("<b>", "")
             .replace("</b>", "").replace(",", "")
             for c in line.split("|")]
    if len(cells) > 16 and re.fullmatch(r"20\d\d", cells[2]):
        months = sum(int(c) for c in cells[3:15])
        checked += 1
        if abs(months - int(cells[15])) > 5:
            print(cells[2], months, cells[15])
print(checked, "rows checked")
# 30 rows checked
```

Week 9's first check: a row's twelve months should add up to its total. Nothing printed, so no row missed by more than $5.

How close

17 of the 30 rows add up exactly; the rest are within $2, because the report rounds each figure to the dollar.

Microsprint: 2018

Its Exhibit 3 is a table of text: 15 pages, 4.5 cents, 28 rows checked, all exact.

What a sum can't catch

Week 9's Docling read 21 of these 30 rows whole, and put two of them beside the wrong label. Those rows passed the sums.

Step 5 saves the Markdown to a file, so Wednesday doesn't pay to read it again.

2. Wednesday · Live-Coding Lab

Today: Part 2, a model as a function

A language model, called like any other API: an instruction and one page in, JSON in a shape we define out. Then we check it against week 9.

- Find the page with the summary table's last row
- Ask for its totals as JSON, with a schema
- Check them against pdfplumber's figures; then do all seven reports

Use it | Steps 6–10

- Download the new `lab-13.ipynb`; start at **Step 6**
- Only `OPENROUTER_KEY`: we read the book's saved Markdown of all seven reports, made as on Monday

Free models are shared

Models whose ID ends in `:free` cost nothing, and a whole class shares them. When one is busy it answers `429`. We send a list of `models`, and OpenRouter tries the next.

OpenRouter

One kind of request, to models from many companies. It also adds one more company to the list of who sees your documents.

Steps 7–8: one page, and the shape of the answer

```
markdown = requests.get(
    saved + "revenue-report-2024-12.md",
    headers=HEADERS).text
pages = re.split(r"\n*\{\d+\}-{10,}\n*",
                 markdown)[1:]
summary = [page for page in pages
           if "Total Sales and Use Tax" in page][0]
```

That one page, 4,902 characters, is all the model sees.

The instruction

"You extract figures from one page of a City of Boulder revenue report. Copy numbers exactly as printed, without $ or commas. Use only the page."

```
# Given: the answer's shape, in JSON Schema
SCHEMA = {
  "type": "object",
  "properties": {
    "month": {"type": "string", "enum": MONTHS},
    "year": {"type": "integer"},
    "total_prior": {"type": "integer",
      "description": "The table's last row, "
        "Total Sales and Use Tax, for the "
        "earlier year, in dollars"},
    "total_current": {"type": "integer", ...},
  },
  "required": ["month", "year",
               "total_prior", "total_current"],
  "additionalProperties": False,
}
```

In the lab, every field has a description too. `enum` lists the twelve months the model may give.

Step 8: one request, one JSON answer

```
MODELS = ["nvidia/nemotron-3.5-super-vl:free",
          "google/gemma-4-31b-it:free",
          "dots-studio/dots-3-note-preview:free"]
body = {"models": MODELS, "temperature": 0,
  "messages": [
    {"role": "system", "content": INSTRUCTION},
    {"role": "user", "content": summary}],
  "response_format": {"type": "json_schema",
    "json_schema": {"name": "totals", "strict": True,
                    "schema": SCHEMA}}}
response = requests.post(
    f"{openrouter_base}/chat/completions", json=body,
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
```

Docs: OpenRouter's structured outputs and model fallbacks; free models checked November 7

Each part of the body

- `messages`: a `system` instruction and a `user` page. That's the whole conversation; we never add a third.
- `response_format`: answer in `SCHEMA`'s shape. `strict` says no other fields.
- `temperature` 0: its most likely answer.
- `models`: if the first is busy, try the next. Embeddings get no list (Friday).

Step 9: the answer, and the check

```
print(response["model"])
answer = json.loads(
    response["choices"][0]["message"]["content"])
print(answer)
# nvidia/nemotron-3.5-super-vl:free
# {'month': 'December', 'year': 2025,
#  'total_prior': 173908431, 'total_current': 171006199}

prior, current = TRUTH["revenue-report-2024-12"]
print("prior:  ", answer["total_prior"], prior,
      answer["total_prior"] == prior)
print("current:", answer["total_current"], current,
      answer["total_current"] == current)
# prior:   173908431 175517129 False
# current: 171006199 173908431 False
```

Valid JSON, three fields wrong

Our first request, on November 7:

- `total_prior` is 2024's total, not 2023's
- `total_current` is 2024's *subtotal*, the row above: "based on current economic activity"
- 2025 is from "received by the city through January 2025"

`TRUTH` is week 9's pdfplumber figures for the last row. Without them, would you have noticed?

`usage`: 1,568 tokens in, 2,080 out, 2,027 of them `reasoning_tokens`. Cost: 0.

The same request, again: non-determinism

```
for attempt in range(3):
    response = requests.post(
        f"{openrouter_base}/chat/completions",
        json=body, headers=...).json()
    print(json.loads(
        response["choices"][0]["message"]["content"]))
    time.sleep(2)
# {'month': 'December', 'year': 2025, 'total_prior':
#  175517129, 'total_current': 173908431}
# {'month': 'December', 'year': 2025, 'total_prior':
#  175517129, 'total_current': 173908431}
# {'month': 'December', 'year': 2025, 'total_prior':
#  175517129, 'total_current': 173908431}
```

Same body, same model, `temperature` 0, a minute later: both totals right, the year still wrong.

The 2024 report, six requests

Request	Totals	Year
Step 8	wrong	2025
Microsprint, 3	right	2025
Step 10, twice	right	2025

November 7, 22:01–22:07 UTC; every answer from the same free model.

Why?

`temperature` 0 asks for the most likely answer, but shared hardware can change the arithmetic slightly, and 2,000 tokens of reasoning give a small change room to grow. Earlier that day, the chapter's checks got 2023, then 2024.

Step 10: seven reports

```
rows = []
for report, (prior, current) in TRUTH.items():
    pages = re.split(r"\n*\{\d+\}-{10,}\n*",
        requests.get(saved + report + ".md",
                     headers=HEADERS).text)[1:]
    summary = [page for page in pages
        if "Total Sales and Use Tax" in page][0]
    answer, model = extract_totals(summary)
    rows.append({"report": report, **answer,
        "prior_ok": answer["total_prior"] == prior,
        "current_ok":
            answer["total_current"] == current})
    time.sleep(2)
results = pd.DataFrame(rows)
```

`extract_totals(page)` is Step 8's request in a function: we send it seven times.

Both totals right? Two runs

Report	Run 1	Run 2	Run 2 said
2018	right	**wrong**	December 2018
2019	**wrong**	**wrong**	January 2020
2020	right	right	January 2021
2021-12	right	right	December 2021
2022-12	right	right	December 2022
2023-12	right	right	December 2023
2024-12	right	right	December 2025
	6 of 7	5 of 7	

Two runs a few minutes apart, November 7.

Every report covers through December of its own year. The month or the year is wrong in four of seven.

What went wrong: one mistake, three times

The 2019 report's summary page, its last two rows:

	2018	2019
Total … current economic activity	$139,292,879	$139,548,151
Total Sales and Use Tax	$140,609,751	$141,954,650

The model's answer, both runs:
`total_prior`: 141954650 `total_current`: 139548151

It took the report year's total for the year before, and the subtotal above it for the report year. 2018's second run, and Step 8's 2024 request, made the same move.

Not a missing row

Both pages hold the whole table, the total row included. The model had the right figures in front of it.

Months and years

"January" and the next year come from "received by the city through January 2019".

Discuss

A better model, a better instruction, or a better page: which would you try first, and how would you know it worked?

3. Friday · Live-Coding Lab

Today: Part 3, search by meaning, answer with sources

Seven reports make 243 passages. Which ones answer a question? Then a model answers from those passages only, as JSON with the IDs of its sources, or null.

Lab | Steps 11–15

- Cut the reports into chunks; embed them
- Search by meaning, with cosine similarity
- Answer with sources: retrieval-augmented generation, kept to a figure
- How MCP and skills describe tools to a model
- Download the new `lab-13.ipynb`; start at **Step 11**

Then | your project, 15 minutes

- Portfolio checkpoint 3, *Analyze your data*

Embeddings

An embedding model turns a text into a list of numbers, so that texts about the same thing are close together. A question becomes a list of numbers too; the closest passages are the ones to read.

Still only `OPENROUTER_KEY`.

Steps 11–12: chunks, and one embedding

Given: each page cut into chunks of at most about 2,500 characters, between lines, each with an ID.

```
# 243 chunks from 7 reports

EMBEDDER = "nvidia/nemotron-3-embed-1b:free"
one = requests.post(f"{openrouter_base}/embeddings",
    json={"model": EMBEDDER,
          "input": [chunks["text"][0]]},
    headers={"Authorization": f"Bearer {openrouter_key}",
             **HEADERS}).json()
print(len(one["data"][0]["embedding"]), "numbers")
# 2048 numbers
```

Then a given loop embeds all 243, 20 to a request, and scales each vector to length 1: `(243, 2048)`.

Docs: OpenRouter's embeddings endpoint

Why chunks?

The embedding model reads at most 4,096 tokens. Exhibit 3's page is 16,333 characters of padded table. A chunk's ID, such as `2024-12 p.9 #1`, says where it came from: the report, the page, the piece.

One embedding model

Vectors from two models live in different spaces and can't be compared. So `EMBEDDER` gets no fallback list, and a new model means embedding everything again.

Step 12: cosine similarity, and what it finds

```
question = ("How much revenue did the state share "
            "back from marijuana taxes?")
response = requests.post(f"{openrouter_base}/embeddings",
    json={"model": EMBEDDER, "input": [question]},
    headers=...).json()
query = np.array(response["data"][0]["embedding"])
query = query / np.linalg.norm(query)
chunks["score"] = vectors @ query
print(chunks.nlargest(5, "score")[["id", "score"]])
#                  id     score
# 41      2019 p.8 #1  0.467471
# 75      2020 p.9 #1  0.466428
# 9       2018 p.9 #1  0.463676
# 187  2023-12 p.9 #1  0.460978
# 152  2022-12 p.9 #1  0.458975
```

For vectors of length 1, cosine similarity is the dot product, so `vectors @ query` scores all 243 at once.

Topics, not figures

The marijuana page from five years, within 0.01 of each other, and not 2024's.

"Total sales and use tax in 2022" found 2022's summary page first, but the half without the total row. The half with it came fourth.

So keep one report's chunks before scoring: a filter you can check.

Step 13: an answer with sources, or null

```
top = chunks[chunks["report"] == "revenue-report-2024-12"
             ].nlargest(3, "score")
passages = "\n\n".join(f"{row.id}: {row.text}"
                       for row in top.itertuples())
body = {"models": MODELS, "temperature": 0,
  "messages": [
    {"role": "system", "content": ANSWER_INSTRUCTION},
    {"role": "user",
     "content": f"Question: {question}\n\n{passages}"}],
  "response_format": {"type": "json_schema",
    "json_schema": {"name": "answer", "strict": True,
                    "schema": ANSWER_SCHEMA}}}
response = requests.post(
    f"{openrouter_base}/chat/completions",
    json=body, headers=...).json()
print(response["model"], json.loads(
    response["choices"][0]["message"]["content"]))
# nvidia/nemotron-3.5-super-vl:free
# {'value': 529220, 'source_ids': ['2024-12 p.9 #1']}
```

Given: `ANSWER_SCHEMA`, whose `value` is `["integer", "null"]`, and `source_ids`, a list. The instruction: a figure only if a passage prints it.

Check it

Week 9's pdfplumber finds $529,220 printed on Chart 8, page 9: the source it gave.

Microsprint: the beverage tax

"How much sugar-sweetened beverage tax did the city collect in December 2024?"

```
{'value':  None,
 'source_ids':  ['2024-12 p.10 #1']}
```

Null is right: no page prints it, and `~$285,000` was Datalab's estimate. But the schema said no sources with a null. Check those too.

Step 14: what tools a server offers, by MCP

```
mcp = "https://mcp.deepwiki.com/mcp"
mcp_headers = {**HEADERS,
    "Accept": "application/json, text/event-stream"}
hello = {"jsonrpc": "2.0", "id": 1, "method": "initialize",
  "params": {"protocolVersion": "2025-06-18",
    "capabilities": {},
    "clientInfo": {"name": "info4617", "version": "1.0"}}}
requests.post(mcp, json=hello, headers=mcp_headers)
listing = requests.post(mcp, json={"jsonrpc": "2.0",
    "id": 2, "method": "tools/list"},
    headers=mcp_headers).text
line = [l for l in listing.splitlines()
        if l.startswith("data: ")][0]
for tool in json.loads(line[6:])["result"]["tools"]:
    print(tool["name"], tool["inputSchema"]["required"])
# ask_wiki_question ['repoName', 'question']
# read_wiki_contents ['repoName']
# read_wiki_structure ['repoName']
```

The **Model Context Protocol** is a shared way for a server to say which tools it offers. DeepWiki's public server needs no key. It answers as a short event stream:

```
event: message
data: {"jsonrpc":"2.0","id":2,
  "result":{"tools":[...]}}
```

Each tool's `inputSchema` is **JSON Schema**, Wednesday's language: `"question": {"type": "string"}`.

Discuss: `inputSchema` and `response_format`: which constrains what a model sends, and which what it returns?

Docs: the MCP specification, tools

Step 14: a skill is a Markdown file

```
skill = requests.get("https://raw.githubusercontent.com/"
    "anthropics/skills/main/skills/pdf/SKILL.md",
    headers=HEADERS).text
print("\n".join(skill.splitlines()[:5]))
# ---
# name: pdf
# description: Use this skill whenever the user wants
#   to do anything with PDF files. This includes reading
#   or extracting text/tables from PDFs, ...
# license: Complete terms in LICENSE.txt
# ---
```

The description runs on to merging, splitting, forms and OCR; cut here.

Front matter, then instructions

A **skill** is a folder of instructions. Its `name` and `description` tell a model when to read the rest: 314 lines of Markdown and code.

Its code uses **pypdf** and **pdfplumber**, week 9's tools.

Discuss

If a model followed this skill on Boulder's reports, which of week 9's checks would still be yours to run?

Step 15: what did we learn, and what's due

Answer as comments

- Which of Datalab's output would you publish, and which would you check first?
- Which reports did the model get wrong: was it the model, or what we sent?
- Why did embeddings find topics more easily than figures?
- What would you check your own project's model outputs against?

Due Sunday | Recommended Exercises

From chapter 13's companion notebook, on a public PDF you choose, with no private individuals' names:

1. Convert it: Monday's Steps 1–3
2. Write a schema of your own, and extract
3. Check it: a sum, and a ground truth
4. Ask a question with sources, and one whose answer isn't there
5. Report the costs: pages, cents, requests

Graded on good-faith attempt to complete rather than getting everything working.

Friday's last 15 minutes: portfolio checkpoint 3

Due Sunday, Nov 15 | Analyze your data

- **An analysis notebook** that runs top to bottom, with its output visible
- **Results that answer at least part of your question**: the counts or comparisons your proposal named
- **At least one figure** saved in your repository, with a title, labeled axes, and a one-sentence caption
- **The README's Methods section** and your preliminary **Findings**

Run your analysis notebook once, top to bottom, here, where you can ask. Used a model anywhere? Say in Methods which one, when, and what you checked its output against.

Next week: automating data collection (chapter 14).

Key takeaways

1. Call a model **as a function**: one request in, JSON in your schema out. No conversation.
2. **Valid JSON can still be wrong.** Check it against something you trust: a sum, a second tool, a ground truth.
3. Document APIs **read** some things and **write** others. A chart's description is the model's, not the document's.
4. The same request can give a different answer. **Record** the model, the date, and the raw response.
5. Embeddings find **topics**, not figures; an answer needs its **sources**, and null when there are none.